Bliss Executable Context

Abstract Specification

Mayukh Chakraborty

Version 3.0

Target Environment: Sandboxed (designed for Minecraft mod integration, but portable)

Contents

1 Changelog **3**

2 Overview **3**

3 Components **3**

3.1 Memory (`BlissMemory`) 4

3.2 Registers (`BlissRegisters`) 5

3.3 Stack (`BlissStack`) 5

3.4 Devices (`BlissDevice`) 6

3.5 Messages (`BlissMessage`) 6

3.6 Errors (`BlissError`) 6

3.7 Machine (`BlissMachine`) 7

4 Instruction Format **8**

4.1 Encoded Length and Decode Rule 8

5 Type System **9**

6 Opcode Reference **9**

6.1 Long Arithmetic (64-bit signed) 9

6.2 Int Arithmetic (32-bit signed) 9

6.3 Float Arithmetic (32-bit IEEE 754) . . . 10
6.4 Double Arithmetic (64-bit IEEE 754) . . . 10
6.5 Bit Crops . . . 10
6.6 Type Conversions . . . 11
6.7 Control Flow . . . 11
6.8 Comparisons . . . 12
6.9 Bitwise Operations . . . 12
6.10 Register Operations . . . 12
6.11 Memory Operations . . . 13
6.12 Stack Operations . . . 14
6.13 Device Operations . . . 14
6.14 Exit . . . 15

7 Host Responsibilities **16**

8 Portability Notes **16**

9 Diagrams **16**

9.1 Register / ALU Path . . . 16
9.2 Data Movement . . . 16
9.3 `execute()` Result Flow . . . 16

1 Changelog

Version	Changes
3.0	Variable-length instruction fetch by opcode; instruction slot schema reduced to `4-4-8` (`opcode`, `first_value`, `second_value`); sparse encodings refactored so `JUMP/JUMP_IF/JUMP_IF_NOT` and `LOAD/STORE/ILOAD/ISTORE` use `second_value` for targets or addresses; `execute()` PC commit behavior clarified; added `BlissError` codes `STACK_OVERFLOW` and `STACK_UNDERFLOW`; stack semantics clarified with no modulo wrap and explicit fault behavior; all pop-based opcodes document `STACK_UNDERFLOW`, and all push operations document `STACK_OVERFLOW`.
2.2	Added int-width memory operations: `ILOAD`, `ISTORE`, `ILOAD_REG`, `ISTORE_REG` for compact 32-bit storage while preserving register-level int contract semantics.
2.1	Added dynamic opcode alternatives with explicit operand-source suffixes where needed: `_REG` (operands from `R1/R2`) and `_STK` (operands from stack for selected arity ≥ 3 forms). Legacy unsuffixed opcodes remain supported for backward compatibility.
2.0	Added stack system; added `BlissError`, `BlissMessage`, `BlissDevice`; `execute()` now returns `BlissError`; `EXIT` no longer calls `System.exit()`; `MEMORY_COPY` now stack-driven; device opcodes now stack-driven; `MEMORY_COPY` size sourced from stack; int op result contract clarified.
1.0	Initial specification.

2 Overview

A *Bliss Executable Context* (BEC) is a self-contained virtual machine that executes Bliss bytecode. It is host-driven by design: the host controls construction, program loading, scheduling, and teardown. The machine itself exposes one execution step.

The BEC does not own scheduling. The host decides how often `execute()` is called. Each `execute()` call processes exactly one instruction and then returns. That one instruction may still do variable-time work and may block the calling thread.

The architecture is hybrid register-stack:

- Registers handle fixed-operand math, logic, and control work.
- Stack handles variable-arity argument passing and stack-driven operations.
- Memory links both domains.

3 Components

A conforming BEC implementation provides the following components.

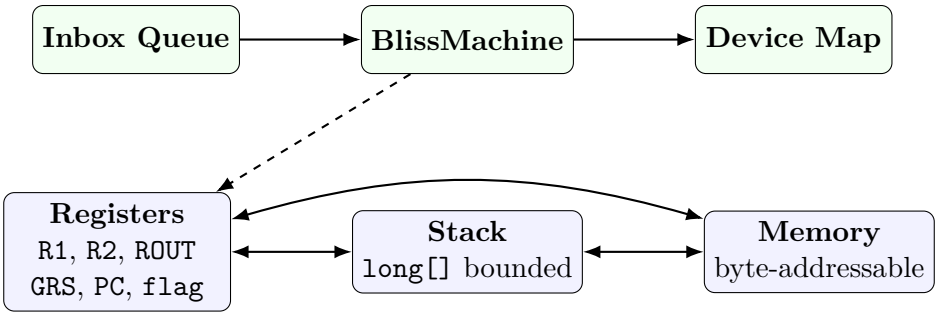

Figure 1: Conceptual architecture of a Bliss Executable Context.

3.1 Memory (`BlissMemory`)

- Flat, byte-addressable memory of fixed size.
- All addresses resolve using *floor-mod* and never go out of bounds.
- Address rule: `resolved = floorMod(address, memorySize)`.
- Java mapping: `Math.floorMod(address, memorySize)`.
- Endianness: big-endian.

Required operations:

Method	**Width**	**Notes**
`getInt(address)`	32 bits	4 bytes, big-endian
`getLong(address)`	64 bits	8 bytes, big-endian
`getFloat(address)`	32 bits	IEEE 754 via `intBitsToFloat`
`getDouble(address)`	64 bits	IEEE 754 via `longBitsToDouble`
`getByte(address)`	8 bits	Payload extraction only
`writeInt(address, value)`	32 bits	—
`writeLong(address, value)`	64 bits	—
`writeFloat(address, value)`	32 bits	via `floatToRawIntBits`
`writeDouble(address, value)`	64 bits	via `doubleToRawLongBits`
`writeByte(address, value)`	8 bits	Payload writes only
`loadProgram(bytes, startAddress)`	n bytes	Copies raw bytecode into memory

Note. `getByte` and `writeByte` exist to support payload I/O and `MEMORY_COPY`. They are not general-purpose instruction operands.

3.2 Registers (`BlissRegisters`)

Register	Type	Purpose
`R1`	`long`	Primary ALU input
`R2`	`long`	Secondary ALU input
`ROUT`	`long`	ALU, convert, and stack output
`GRS[0..7]`	`long[8]`	General-purpose indexed register bank
`PC`	`long`	Program counter (next instruction address)
`flag`	`byte`	Condition register (`0` false, `1` true)

All registers are 64-bit stores. `int`/`float` typed ops read lower 32 bits where applicable.

Register index space for register-addressed opcodes such as `MOVE` and `CLEAR`:

Index	Register
`0`	`R1`
`1`	`R2`
`2`	`ROUT`
`3..10`	`GRS[0..7]`
`11`	`PC`

`flag` is excluded from `MOVE` and `CLEAR`. Use `SET_FLAG` for `flag` writes.

Int result contract. After `IADD`, `ISUB`, `IMUL`, or `IDIV`, `ROUT` stores the 32-bit result sign-extended to 64 bits. No extra conversion is needed to preserve signed numeric meaning. This contract defines register representation only; memory storage width depends on the memory opcode used.

3.3 Stack (`BlissStack`)

- Fixed-size `long` array.
- Single integer `top` pointer.
- Overflow and underflow are faults (`STACK_OVERFLOW`, `STACK_UNDERFLOW`).

```
BlissStack {
    long[]  array        // fixed storage
    int     top          // next write index (0..size)
    int     size         // stack capacity
}
```

`top` is the next write position:

- `STACK_PUSH`: if `top == size`, return `STACK_OVERFLOW`; otherwise write `array[top]`, then `top = top + 1`.
- `STACK_POP`: if `top == 0`, return `STACK_UNDERFLOW`; otherwise `top = top - 1`, then read `array[top]`.

Note. Behavior for `size <= 0` is not explicitly defined in this abstract spec. In Minecraft host usage, CPU tiers define hard-coded positive stack sizes.

Stack data movement directions:

Direction	Opcodes
Register → Stack	`STACK_PUSH`
Stack → Register	`STACK_POP`
Stack → Memory	`STACK_STORE` (address and byte-size from registers)

3.4 Devices (`BlissDevice`)

Devices are abstract I/O endpoints stored in a map keyed by integer ID. The host populates this map before execution.

```
BlissDevice {
    abstract void send(byte[] data, int size)
}
```

3.5 Messages (`BlissMessage`)

```
BlissMessage {
    byte[]  data
    int     size
}
```

Inbound messages arrive through a host-managed `Queue<BlissMessage>`. BEC reads via `READ_MESSAGE`; it never writes to inbox.

3.6 Errors (`BlissError`)

The VM returns faults as values from `execute()`. It does not throw in-program execution faults.

Field	Type	Meaning
`code`	`BlissError.Code`	Enumerated result or fault type
`message`	`String`	Human-readable detail
`pc`	`long`	Faulting instruction address (`PC` before fetch increment)

Error codes:

Code	Meaning
NONE	Success (BlissError.NONE singleton)
HALT	Clean program exit via EXIT
UNKNOWN_OPCODE	Opcode out of range or unrecognized
DEVICE_NOT_FOUND	SEND_MESSAGE referenced unknown device ID
DIVISION_BY_ZERO	Integer divide-by-zero (IDIV, LDIV)
INVALID_OPERAND	Structurally invalid operand, for example a negative size
STACK_OVERFLOW	Stack push attempted when stack has no free slot
STACK_UNDERFLOW	Stack pop attempted when stack is empty

3.7 Machine (BlissMachine)

Constructor dependencies:

- BlissRegisters registers
- BlissMemory memory
- BlissStack stack
- Map<Integer, BlissDevice> devices
- Queue<BlissMessage> inbox

Execution entry:

```
BlissError execute()
```

Per-call behavior (one instruction):

1. Capture faultPc = registers.PC.
2. Fetch opcode (4 bytes) at faultPc.
3. Initialize nextPc = faultPc + 4.
4. Validate opcode bounds (UNKNOWN_OPCODE on failure).
5. Decode only the slots required by that opcode (first_value, second_value) and update nextPc to the encoded instruction length.
6. Execute opcode logic.
7. Before returning from non-control-transfer paths, commit registers.PC = nextPc.
8. Control-transfer opcodes may overwrite PC directly, for example JUMP or a taken JUMP_IF.
9. Return BlissError.NONE or a fault result.

The host repeatedly calls execute() according to scheduling policy.

4 Instruction Format

Instruction slots use a fixed schema with a maximum footprint of 16 bytes (128 bits):

```
[ opcode       : 4 bytes (int)  ]
[ first_value  : 4 bytes (int)  ]
[ second_value : 8 bytes (long) ]
```

Encoding is variable-length: opcodes encode only the slots they require.

Field	Type	Typical role
`opcode`	`int`	Operation selector (enum index)
`first_value`	`int`	Instruction-operand slot (destination index, bit count, flag value)
`second_value`	`long`	Instruction-operand slot (source index, bit start)

4.1 Encoded Length and Decode Rule

Encoded length profile:

Profile	Encoded bytes	Binary slot form
Opcode only	4	`[opcode]`
Opcode + first	8	`[opcode][first_value]`
Opcode + first + second	16	`[opcode][first_value][second_value]`

Rules:

- Decoder starts with `opcode` at `PC`, then fetches additional slots only if required by opcode semantics.
- `PC` step for non-control-transfer return equals the selected encoded profile length.
- Unused-but-encoded reserved slots must not fault; writers should encode them as zero.

Examples:

- `IADD`: `[IADD]` (4 bytes)
- `BIT_CROP`: `[BIT_CROP][length][start]` (16 bytes)
- `LOAD`: `[LOAD][to_register][from_mem_address]` (16 bytes)

Unused fields are ignored. Non-zero unused values must not fault.

Stack-driven opcodes such as `MEMORY_COPY`, `SEND_MESSAGE`, `READ_MESSAGE`, `LIST_DEVICES`, and `BIT_CROP_STK` consume operands from stack and encode as opcode-only in this version. Some of these are legacy unsuffixed opcodes retained for compatibility.

Dynamic variant naming for opcodes that define a dynamic alternative:

- `_REG`: operands are read from registers (`R1` then `R2` for one- and two-operand forms).

- `_STK`: operands are read from stack (used for selected arity ≥ 3 forms).

Not all stack-driven operations use a `_STK` suffix. Several unsuffixed stack-driven opcodes are legacy forms kept stable for compatibility.

5 Type System

Type ladder:

```
int (32-bit signed) -> long (64-bit signed) -> float (32-bit IEEE 754) -> double (64-bit
    IEEE 754)
```

Rules:

- `byte` and `short` have no arithmetic opcodes; they are storage forms only.
- Any `byte`/`short` loaded into registers is zero-extended and treated as `int`.
- Float and double values are stored as raw bits inside `long` register or stack slots.
- Conversions preserve NaN payloads through raw-bit APIs.
- IEEE float and double divide-by-zero yields `Infinity` or `NaN` with no fault.
- Integer divide-by-zero returns `DIVISION_BY_ZERO`.

6 Opcode Reference

Each opcode table includes an explicit in-memory format in the **Format** column.

6.1 Long Arithmetic (64-bit signed)

Operands: `R1`, `R2` → `ROUT`

Opcode	Format	Operation
`LADD`	`[LADD]`	`ROUT = R1 + R2`
`LSUB`	`[LSUB]`	`ROUT = R1 - R2`
`LMUL`	`[LMUL]`	`ROUT = R1 * R2`
`LDIV`	`[LDIV]`	`ROUT = R1 / R2`; returns `DIVISION_BY_ZERO` if `R2 == 0`

6.2 Int Arithmetic (32-bit signed)

Operands: `(int)R1`, `(int)R2` → `ROUT` (sign-extended to 64 bits)

Opcode	Format	Operation
IADD	[IADD]	ROUT = (int)R1 + (int)R2
ISUB	[ISUB]	ROUT = (int)R1 - (int)R2
IMUL	[IMUL]	ROUT = (int)R1 * (int)R2
IDIV	[IDIV]	ROUT = (int)R1 / (int)R2; returns DIVISION_BY_ZERO if (int)R2 == 0

6.3 Float Arithmetic (32-bit IEEE 754)

Lower 32 bits of R1/R2 are interpreted as float, and the result is stored as raw bits in ROUT.

Opcode	Format	Operation
FADD	[FADD]	ROUT = floatBits(floatOf(R1) + floatOf(R2))
FSUB	[FSUB]	ROUT = floatBits(floatOf(R1) - floatOf(R2))
FMUL	[FMUL]	ROUT = floatBits(floatOf(R1) * floatOf(R2))
FDIV	[FDIV]	ROUT = floatBits(floatOf(R1) / floatOf(R2)); divide-by-zero yields Infinity/NaN and does not fault

6.4 Double Arithmetic (64-bit IEEE 754)

R1/R2 are interpreted as raw double bits, and the result is stored as raw bits in ROUT.

Opcode	Format	Operation
DADD	[DADD]	ROUT = doubleBits(doubleOf(R1) + doubleOf(R2))
DSUB	[DSUB]	ROUT = doubleBits(doubleOf(R1) - doubleOf(R2))
DMUL	[DMUL]	ROUT = doubleBits(doubleOf(R1) * doubleOf(R2))
DDIV	[DDIV]	ROUT = doubleBits(doubleOf(R1) / doubleOf(R2)); divide-by-zero yields Infinity/NaN and does not fault

6.5 Bit Crops

Operands:

- Legacy forms use R1 plus instruction fields first_value and second_value.
- Dynamic forms use either _REG variants or stack-driven _STK variants.

Validation:

- RIGHT_BIT_CROP/LEFT_BIT_CROP and their _REG variants require N in $[0, 64]$.
- BIT_CROP and BIT_CROP_STK require length in $[0, 64]$, start in $[0, 63]$, and start + length <= 64.
- Violations return INVALID_OPERAND.

Opcode	Format	Operation
RIGHT_BIT_CROP	[RIGHT_BIT_CROP][N]	ROUT = (N == 64) ? 0 : (R1 & ((1L « N) - 1L))
LEFT_BIT_CROP	[LEFT_BIT_CROP][N]	ROUT = (N == 0) ? R1 : (R1 & ((1L « (64 - N)) - 1L))
BIT_CROP	[BIT_CROP][length][start]	ROUT = (length == 64) ? R1 : ((R1 »> start) & ((1L « length) - 1L))
RIGHT_BIT_CROP_REG	[RIGHT_BIT_CROP_REG]	Uses R1 as value and R2 as N; same crop formula as RIGHT_BIT_CROP; output to ROUT
LEFT_BIT_CROP_REG	[LEFT_BIT_CROP_REG]	Uses R1 as value and R2 as N; same crop formula as LEFT_BIT_CROP; output to ROUT
BIT_CROP_STK	[BIT_CROP_STK]	Pops length (top), start, value (bottom); applies the bit-crop formula and writes ROUT. If stack has fewer than 3 values, return STACK_UNDERFLOW.

6.6 Type Conversions

Operand: R1 → ROUT

Opcode	Format	Operation
INT_TO_LONG	[INT_TO_LONG]	Sign-extend lower 32 bits
INT_TO_FLOAT	[INT_TO_FLOAT]	(float)(int)R1 stored as raw bits
INT_TO_DOUBLE	[INT_TO_DOUBLE]	(double)(int)R1 stored as raw bits
LONG_TO_INT	[LONG_TO_INT]	Truncate to low 32 bits
LONG_TO_FLOAT	[LONG_TO_FLOAT]	(float)R1 stored as raw bits
LONG_TO_DOUBLE	[LONG_TO_DOUBLE]	(double)R1 stored as raw bits
FLOAT_TO_INT	[FLOAT_TO_INT]	(int)floatOf(R1) toward zero
FLOAT_TO_LONG	[FLOAT_TO_LONG]	(long)floatOf(R1) toward zero
FLOAT_TO_DOUBLE	[FLOAT_TO_DOUBLE]	Widen float to double and store raw bits
DOUBLE_TO_INT	[DOUBLE_TO_INT]	(int)doubleOf(R1) toward zero
DOUBLE_TO_LONG	[DOUBLE_TO_LONG]	(long)doubleOf(R1) toward zero
DOUBLE_TO_FLOAT	[DOUBLE_TO_FLOAT]	Narrow double to float and store raw bits

6.7 Control Flow

Jump target source depends on opcode variant.

Opcode	Format	Operation
JUMP	[JUMP][reserved][target_address]	PC = second_value
JUMP_IF	[JUMP_IF][reserved][target_address]	If flags != 0, then PC = second_value

Opcode	Format	Operation
JUMP_IF_NOT	[JUMP_IF_NOT][reserved][target_address]	If flag == 0, then PC = second_value
JUMP_REG	[JUMP_REG]	PC = R1
JUMP_IF_REG	[JUMP_IF_REG]	If flag != 0, then PC = R1
JUMP_IF_NOT_REG	[JUMP_IF_NOT_REG]	If flag == 0, then PC = R1

6.8 Comparisons

Operands: R1, R2 → flag (1 true, 0 false)

Opcode	Format	Condition
EQUALS	[EQUALS]	R1 == R2 (bitwise and type-agnostic)
LONG_LESS_THAN	[LONG_LESS_THAN]	R1 < R2
LONG_LESS_THAN_EQUAL	[LONG_LESS_THAN_EQUAL]	R1 <= R2
INT_LESS_THAN	[INT_LESS_THAN]	(int)R1 < (int)R2
INT_LESS_THAN_EQUAL	[INT_LESS_THAN_EQUAL]	(int)R1 <= (int)R2
FLOAT_LESS_THAN	[FLOAT_LESS_THAN]	floatOf(R1) < floatOf(R2); NaN evaluates to false
FLOAT_LESS_THAN_EQUAL	[FLOAT_LESS_THAN_EQUAL]	floatOf(R1) <= floatOf(R2); NaN evaluates to false
DOUBLE_LESS_THAN	[DOUBLE_LESS_THAN]	doubleOf(R1) < doubleOf(R2); NaN evaluates to false
DOUBLE_LESS_THAN_EQUAL	[DOUBLE_LESS_THAN_EQUAL]	doubleOf(R1) <= doubleOf(R2); NaN evaluates to false

6.9 Bitwise Operations

Operands: R1, R2 except BITWISE_NOT (uses R1 only) → ROUT

Opcode	Format	Operation
BITWISE_AND	[BITWISE_AND]	ROUT = R1 & R2
BITWISE_OR	[BITWISE_OR]	ROUT = R1 | R2
BITWISE_XOR	[BITWISE_XOR]	ROUT = R1 ^ R2
BITWISE_NOT	[BITWISE_NOT]	ROUT = ~R1
BIT_SHIFT_LEFT	[BIT_SHIFT_LEFT]	ROUT = R1 « (R2 & 63)
BIT_SHIFT_RIGHT	[BIT_SHIFT_RIGHT]	ROUT = R1 »> (R2 & 63) (logical right shift)

6.10 Register Operations

Opcode	Operands	Format	Operation
MOVE	first_value dst, second_value src	[MOVE][dst_index][src_index]	Copy between registers in the index space from Section 2.2 (R1, R2, ROUT, GRS[0..7], PC). flag is not addressable by MOVE.
CLEAR	first_value index	[CLEAR][register_index]	Set addressed register to 0 using the same index space as MOVE, excluding flag.
SET_FLAG	first_value value	[SET_FLAG][flag_value]	flag = (byte)(first_value & 0xFF)
MOVE_REG	R1 dst index, R2 src index	[MOVE_REG]	Same register index space and behavior as MOVE, but both indices come from registers.
CLEAR_REG	R1 index	[CLEAR_REG]	Same register index space and behavior as CLEAR, but the index comes from R1.
SET_FLAG_REG	R1 value	[SET_FLAG_REG]	flag = (byte)(R1 & 0xFF)

6.11 Memory Operations

Memory is byte-addressable, big-endian, and floor-mod resolved.

Opcode	Operands	Format	Operation
LOAD	first_value = GR index, second_value = address	[LOAD][to_register][from_memory_address]	GRS[first_value] = memory.getLong(second_value)
STORE	first_value = GR index, second_value = address	[STORE][from_register][to_memory_address]	memory.writeLong(second_value, GRS[first_value])
LOAD_REG	R1 = GR index, R2 = address	[LOAD_REG]	GRS[(int)R1] = memory.getLong(R2)
STORE_REG	R1 = GR index, R2 = address	[STORE_REG]	memory.writeLong(R2, GRS[(int)R1])
ILOAD	first_value = GR index, second_value = address	[ILOAD][to_register][from_memory_address]	GRS[first_value] = signExtend32(memory.getInt(second_value))
ISTORE	first_value = GR index, second_value = address	[ISTORE][from_register][to_memory_address]	memory.writeInt(second_value, (int)GRS[first_value])

Opcode	Operands	Format	Operation
ILOAD_REG	R1 = GR index, R2 = address	[ILOAD_REG]	GRS[(int)R1] = signExtend32(memory.getInt(R2))
ISTORE_REG	R1 = GR index, R2 = address	[ISTORE_REG]	memory.writeInt(R2, (int)GRS[(int)R1])
MEMORY_COPY	Stack: size (top), dest, src (bottom)	[MEMORY_COPY]	Pop size, dest, and src. If stack has fewer than 3 values, return STACK_UNDERFLOW. If size < 0, return INVALID_OPERAND. If size == 0, no-op. Otherwise copy size bytes one-by-one from src to dest with floor-mod address resolution.

MEMORY_COPY push order:

```
STACK_PUSH   // R1 = src address  (first)
STACK_PUSH   // R1 = dest address (second)
STACK_PUSH   // R1 = size bytes   (third, top)
MEMORY_COPY  // pops: size, dest, src
```

6.12 Stack Operations

Opcode	Format	Operation
STACK_PUSH	[STACK_PUSH]	Push R1. If the stack is full, return STACK_OVERFLOW.
STACK_POP	[STACK_POP]	Pop top into ROUT. If the stack is empty, return STACK_UNDERFLOW.
STACK_STORE	[STACK_STORE]	Register-driven bulk write from stack to memory. R1 is the start address and R2 is the byte size. Write in 8-byte chunks: pop one stack value per chunk and write to memory[R1 + offset] for offset = 0, 8, 16, ... until R2 bytes are written. If R2 < 0 or R2 % 8 != 0, return INVALID_OPERAND. If R2 == 0, no-op. If required pops exceed available values, return STACK_UNDERFLOW. Addresses resolve with floor-mod.

Note. Any opcode that pops from stack may return STACK_UNDERFLOW. Any opcode that pushes to stack may return STACK_OVERFLOW.

6.13 Device Operations

Device operations are fully stack-driven and encode as opcode-only in this version.

Opcode	Stack consumed (top → bottom)	Format	Operation
SEND_MESSAGE	size, payload_address, device_id	[SEND_MESSAGE]	If stack has fewer than 3 values, return STACK_UNDERFLOW. If size < 0 or size > Integer.MAX_VALUE, return INVALID_OPERAND. Otherwise read size bytes at payload address and call devices.get(device_id).send(payload, size). If the device ID is missing, return DEVICE_NOT_FOUND.
READ_MESSAGE	dest_address	[READ_MESSAGE]	If stack has fewer than 1 value, return STACK_UNDERFLOW. Poll the inbox. If a message exists, copy its payload to memory at dest_address and set ROUT = payload size; if empty, set ROUT = -1.
LIST_DEVICES	dest_address	[LIST_DEVICES]	If stack has fewer than 1 value, return STACK_UNDERFLOW. Write all known device IDs as consecutive longs to memory at dest_address; set ROUT = count.

SEND_MESSAGE push order:

```
STACK_PUSH   // R1 = device ID       (first)
STACK_PUSH   // R1 = payload address (second)
STACK_PUSH   // R1 = size bytes      (third, top)
SEND_MESSAGE // pops: size, payload_address, device_id
```

6.14 Exit

Opcode	Format	Operation
EXIT	[EXIT]	Return a BlissError with code HALT. The exit code (int)(R1 & 0xFF) is placed in the error message. It must not call System.exit(). The host must stop scheduling after HALT.

7 Host Responsibilities

Concern	Host responsibility
Loading	Call `memory.loadProgram(bytes, startAddress)` and initialize `registers.PC` before the first `execute()`
Scheduling	Choose how many `execute()` calls occur per tick; run on a dedicated thread if needed
Stack sizing	Provide stack size at `BlissStack` construction (for example, from CPU tier in-game)
Pause or resume	Stop or resume calling `execute()`
Teardown	Discard `BlissMachine`; no cleanup API required
Device registration	Populate the device map before running
Inbox delivery	Push `BlissMessage` into the inbox from external systems
Error handling	Inspect return from each `execute()`; stop on `HALT`; log or handle others

8 Portability Notes

- Instruction slot schema is language-agnostic: `4-4-8` big-endian with variable-length encoding (4, 8, or 16 bytes).
- The numeric model maps to common 32-bit and 64-bit integer plus IEEE float platforms.
- Raw-bit float conversions map naturally in Java, C, Python, and JavaScript.
- The device surface is a one-method callback.
- The inbox is a plain FIFO queue.
- The stack is a fixed array plus pointer.
- The fault model is value-based and requires no exception machinery.

9 Diagrams

9.1 Register / ALU Path

9.2 Data Movement

9.3 `execute()` Result Flow

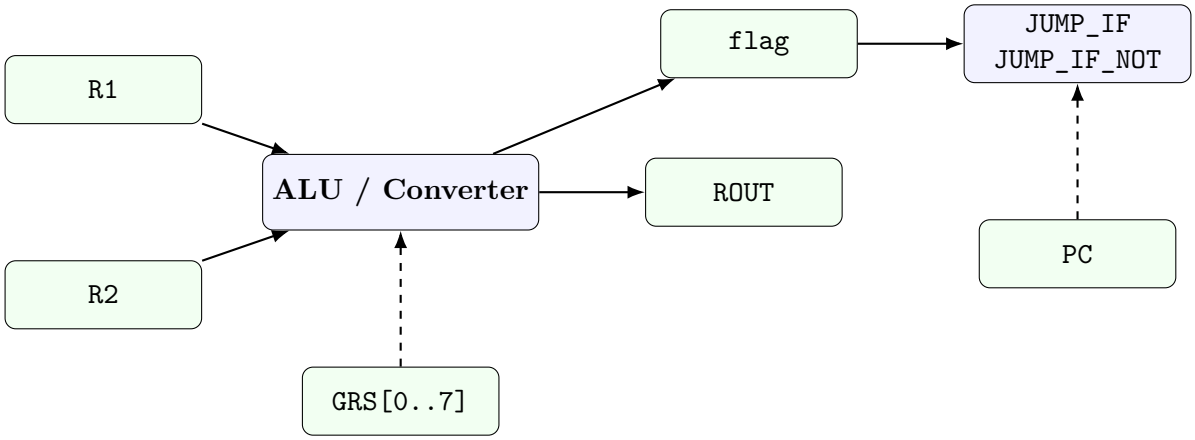

Figure 2: Register and control-flow datapath.

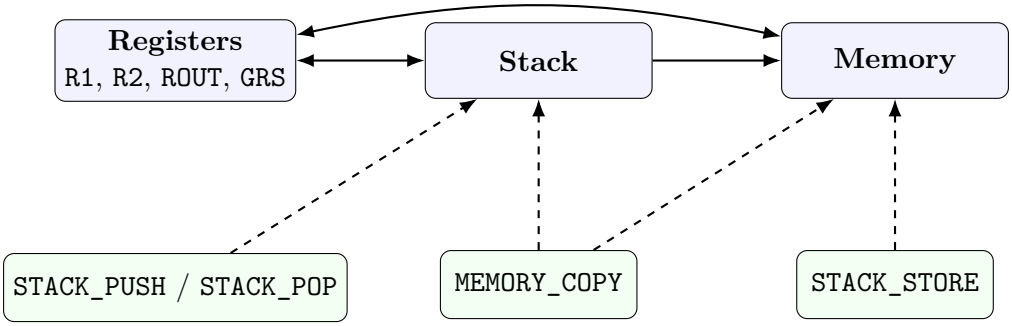

Figure 3: Primary data movement paths between registers, stack, and memory.

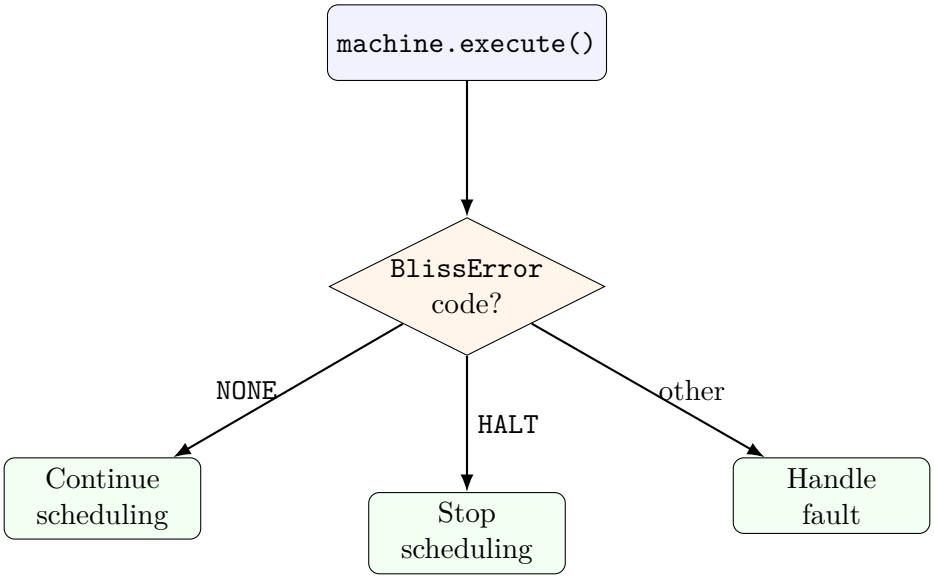

Figure 4: Host-side interpretation of the result returned by `execute()`.